

COLLEGE CODERS

React JS Lesson 15 – API Fetch Flow (Premium Notes)

1) Component Load Process

- When component loads, React renders UI first.
- After first render, side effects can run.

Example:

```
function App(){  
  return <h1>Loading...</h1>  
}
```

Explanation:

- Initial UI appears before API data.

2) useEffect Execution

- useEffect runs after component render.
- Used to trigger API request.

Example:

```
useEffect(()=>{  
  console.log("Component Loaded");  
},[]);
```

Explanation:

- Empty dependency means run once.

3) Sending API Request

- fetch() sends request to external server.

Example:

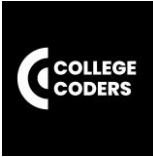
```
fetch('https://jsonplaceholder.typicode.com/users')
```

Explanation:

- Browser requests data from API.

4) Receiving Data from API

- Response must be converted to JSON.



COLLEGE CODERS

Example:

```
.then(res => res.json())
```

Explanation:

- `json()` converts response into usable JavaScript data.

5) Storing Data in State

- Received data is stored using `useState`.

Example:

```
const [users,setUsers] = useState([]);
```

```
setUsers(data);
```

Explanation:

- State stores full API result.

6) Rendering Data in UI

- Stored array is displayed using `map()`.

Example:

```
{users.map((item)=>(  
  <h2 key={item.id}>{item.name}</h2>  
))}
```

Explanation:

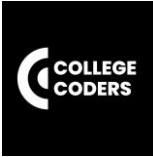
- Each user becomes one UI line.

Mini Project – User List API

```
import { useEffect, useState } from 'react';
```

```
function App(){  
  const [users,setUsers] = useState([]);
```

```
  useEffect(()=>{  
    fetch('https://jsonplaceholder.typicode.com/users')  
      .then(res=>res.json())  
      .then(data=>setUsers(data));  
  })
```



COLLEGE CODERS

```
},[]);
```

```
return (  
  <div>  
    {users.map(user=>(  
      <h2 key={user.id}>{user.name}</h2>  
    ))}  
  </div>  
)  
}
```

- Component loads first.
- `useEffect` runs once.
- API data comes and stores in state.
- `map()` renders all users.